Multithreading:

Multithreading means perform multiple thread simultaneously in waiting of each other called as multithreading

Q. What is thread?

Thread is a subpart of process

Q. What is the process?

Process is a running program in called as process

Note: multithreading is a concept of operating system and java is language which is responsible to implement multithreading and java has two ways to implement multithreading

a. **Using Thread class**
b. **Using Runnable interface**

How to create Thread using a Thread class

a. **Add java.lang package**

   import java.lang.*;

b. **Create class and inherit the Thread class in it**

   public class classname extends Thread
   {
   }

   Example:

   class MyThread extends Thread
   {
   }

   Note: MyThread contain all properties thread class means MyThread is thread

c. **Override its run() method and write thread logics**

   Note: run() is not a method of Thread class basically it is a method of Runnable interface and Runnable interface implemented by Thread class internally
   Purpose of run() method is to write thread logics only
   So any developer want to write thread logics then he can override run() method via Thread class or directly using a Runnable interface and write own custom logics

   Important point:

   Runnable is a functional interface in JAVA because it contain only one abstract metod

Internal Hierarchy of Thread implementation

```
interface Runnable
{
   public void run();
}
class Thread implements Runnable{

}
```

d. Create object of thread child class

```
class MyThread extends Thread
{
    public void run()
          {
             try{
                    for(int i=1; i<=5; i++)
                    {
                      System.out.println("I is  "+i);

                    }
                   }
                   catch(Exception ex)
                   { System.out.println("Error is "+ex);
                   }
          }
}
```

e. Use thread class method for manage thread activity

If we think about Thread class it contain some inbuilt methods which help us to manage overall activity of thread

Methods of Thread class

a. **void start():** this method is used for start the thread means when we call the start() method internally run() method get executed
b. **static void sleep(int milliseconds):** this method is used for hold execution of thread for specified time period.
c. **boolean isAlive():** this method is used to return the status of thread if thread is in execution return true otherwise return false.
d. **void stop():** this method is used for stop the execution of thread or terminate thread
e. **void join(int milliseconds):** this method is used for implement conditional join and purpose of join method is hold execution of another thread according to time set in join method

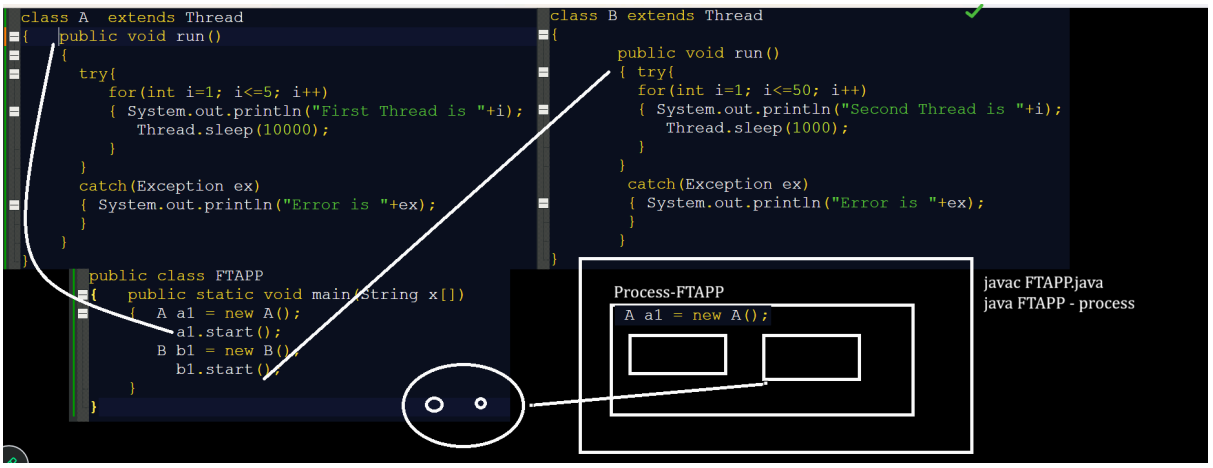
f. **void join():** this method is used for implement unconditional join and purpose of unconditional join() is hold another thread execution whenever running thread is not completed
g. **void setPriority(int priority):** this method is used for set the priority to thread
h. **int getPriority():** this method can return current priority of thread
i. **void setName(String name):** this method can set thread name
j. **String getName():** this method can return thread name
k. **static Thread currentThread():** this method help us return current or working Thread and return reference of Thread object

Etc

Example: we want to create thread and start thread and maintain 1 second waiting period in every iteration of thread

```
class A  extends Thread
{   public void run()
          {
           try{
              for(int i=1; i<=5; i++)
                    { System.out.println("First Thread is "+i);
                          Thread.sleep(10000);
                    }
           }
           catch(Exception ex)
           { System.out.println("Error is "+ex);
           }
          }
}
class B extends Thread
{
            public void run()
            { try{
              for(int i=1; i<=50; i++)
                    { System.out.println("Second Thread is "+i);
                          Thread.sleep(1000);
                    }
            }
             catch(Exception ex)
             { System.out.println("Error is "+ex);
             }
            }
}
public class FTAPP
{   public static void main(String x[])
    {  A a1 = new A();
              a1.start();
            B b1 = new B();
              b1.start();
```

```
        }
}
```



Synchronization and Asynchronization in Thread

Synchronization is the process of coordinating multiple tasks or processes so that they work together correctly - especially when they share resource like memory ,files or data etc

Or

Synchronization is a process where two or more than two threads use the resource sequentially one by one called as synchronization

Asynchronization: asynchronization means when two or more than two threads use the single resource simultaneously waiting of each other called as asynchronization

Now we will discuss about synchronization

Q. What is the benefit of synchronization?

---

a. To coordinate the execution of threads
b. To ensure a thread complete its task before another thread proceeds
c. To avoid race condition when one thread depends on result of another thread
d. To maintain predictable flow in multithreading

How to implement synchronization practically in JAVA?

If we want to implement synchronization in java we have two ways

a. **Using join() method**
b. **Using synchronized keyword**

Now we want to discuss about the join() method

---

a. The join() method is part of the Thread class in Java.
b. It pauses the execution of current thread until the thread on which join() was called finishes its execution

There are two types of join() method

---

a. **Conditional join:** Conditional join() means working thread pause execution or hold other thread execution for specified time period
   Called as conditional join
   Conditional join() means join with parameter called as conditional join

   Syntax: void join(int milliseconds)throws InterruptedException

b. **Unconditional join :** unconditional join means join not allowed other start execution whenever working thread finish its execution and unconditional join means join() without parameter called as unconditional join
   Syntax: void join()

Demo example for understand the behaviour of join

```
class A extends Thread
{
    public void run()
          {
           try{
              for(int i=1; i<=5;  i++)
                    {
                       System.out.println("I is "+i);
                            Thread.sleep(10000);
                    }
           }
           catch(Exception ex)
           { System.out.println("Error is  "+ex);
           }
          }
}
class B extends Thread
{
   public void run()
   { try{
              for(int i=1; i<=50;  i++)
                    {
                       System.out.println("J is "+i);
                            Thread.sleep(1000);
                    }
           }
           catch(Exception ex)
           { System.out.println("Error is  "+ex);
           }
   }
}
public class JoinApplication
```

```
{
    public static void main(String x[])throws Exception
        {
           A a1 =new A();
                a1.start();
                a1.join();
                B b1=new B();
                b1.start();
        }
}
```

Example: Suppose consider we are working on banking application we have two threads Updating the same bank account at the same time.

a. Thread A read the balance (100) and deposit 50 rs and write 150 in database
b. Thread B reads the balance 100 and subtract 30 rs and write in database 120

If both threads their results without synchronization the final balance may be 70 or 150 instead of 120

Example with source code

```
class BankAccount
{
   private int balance=100;

        public void updateBalance(int amount)
        {
           int temp=balance;
          System.out.println(Thread.currentThread().getName()+" reads balance "+balance);
           temp=temp+amount;
                try{
                   Thread.sleep(1000);
                }
                catch(InterruptedException ex)
                { System.out.println(ex);
                }
                balance=temp;
                System.out.println(Thread.currentThread().getName()+" Writes new balance
"+balance);
        }
         public int getBalance()
         {  return balance;
         }
}
class Transaction extends Thread
{
   private BankAccount account;
        private int value;
```

```
        Transaction(BankAccount account,int value)
        {  this.account=account;
           this.value=value;
        }
        public void run()
        { account.updateBalance(value);
        }
}
public class BankApplication
{
    public static void main(String x[])throws Exception
          {
              BankAccount account=new BankAccount();
                    Transaction deposit=new Transaction(account,50);
                     deposit.setName("Deposit Thread");
                    Transaction wid =new Transaction(account,-30);
                     wid.setName("Widraw Thread");

                           deposit.start();
                           deposit.join();
                           wid.start();

                           Thread.sleep(10000);
                           System.out.println("Final Balance is "+account.getBalance());
          }
}
```

How to implement the synchronization by using synchronized keyword

---

```
class Table
{
 synchronized void  showTable(int no)
  {
      try{
              for(int i=1; i<=10; i++)
                    {
                      System.out.printf("%d X %d = %d\n",i,no,i*no);
                           Thread.sleep(1000);
                    }
            }
            catch(Exception ex)
            { System.out.println("Error is "+ex);
            }
  }
}
class Two extends Thread
{  Table table;
   Two(Table table)
```

```
  {  this.table=table;
  }
  public void run()
  {   table.showTable(2);
  }
}
class Three extends Thread
{   Table table;
  Three(Table table)
  {  this.table=table;
  }
  public void run()
  {   table.showTable(3);
  }
}
class Four extends Thread
{   Table table;
  Four(Table table)
  {  this.table=table;
  }
  public void run()
  {   table.showTable(4);
  }
}
public class SyncAsyncApp
{
   public static void main(String x[])throws Exception
        {
           Table t = new Table();
                Two tw = new Two(t);
                tw.start();

                Three th= new Three(t);
                th.start();

                Four f = new Four(t);
                f.start();
        }
}
```

Thread Inter communication

Thread intercommunication means thread can communicate with each other in synchronization state called as thread inter communication
If we want to work with thread intercommunication we have methods of Object class

a. **wait() :** wait method is used for hold thread execution in synchronization
   Basically we have two types of wait

1. **Conditional wait :** conditional wait means hold thread execution for a specified time period and after some time period thread start its working automatically called as conditional wait

   Syntax: void wait(int milliseconds)

2. **Unconditional wait :** unconditional wait means if thread stop its execution never restart execution automatically called as unconditional wait and if we want to restart execution of unconditional wait thread we have to use notify() and notifyAll() method

   Syntax : void wait()

b. **void notify() :** notify() method call only one waited thread in first in first out manner
c. **void notifyAll():** notifyAll() method is used for call all waited threads at time using first in first out format

Example with source code

```
import java.util.*;
class Table
{
 synchronized void  showTable(int no)
  {
      try{
              for(int i=1; i<=10; i++)
                    {
                      System.out.printf("%d X %d = %d\n",i,no,i*no);
                           if(i==5)
                           { wait();
                           }
                           Thread.sleep(1000);
                    }
             }
             catch(Exception ex)
             { System.out.println("Error is "+ex);
             }
  }
  public synchronized void restart(){

            try{
                     notifyAll();
            }
            catch(Exception ex)
            {System.out.println("Error is "+ex);
            }
  }
}
```

```
class Two extends Thread
{  Table table;
   Two(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(2);
   }
}
class Three extends Thread
{   Table table;
   Three(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(3);
   }
}
class Four extends Thread
{   Table table;
   Four(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(4);
   }
}
public class SyncAsyncApp
{
    public static void main(String x[])throws Exception
          {
             Table t = new Table();
                   Two tw = new Two(t);
                   tw.start();

                   Three th= new Three(t);
                   th.start();

                   Four f = new Four(t);
                   f.start();

                   do{
                            Scanner xyz =new Scanner(System.in);
                            String msg=xyz.nextLine();
                            if(msg.equals("restart"))
                            { t.restart();
                            }
                            if(msg.equals("stop"))
```

```
                    {System.exit(0);
                    }
                }while(true);
        }
}
```

Q. What is the difference between wait() and sleep() method?

---

wait	**sleep**
wait() is a method of Object class	Sleep is a static method of Thread class
wait() method can work only in synchronised block	sleep() method can work in synchronized as well as asynchronized block
There are two types of wait a. Conditional wait b. Unconditional wait	sleep() is always work with conditional wait

Q. What is the difference between notify() and notifyAll()?

---

a. notify() method call only one thread or only one waited thread at time in first in first out manner
b. notifyAll() call all waited threads at time in first in first out manner

Q. Can we create a Java program without thread?

No we cannot create java program without thread because every java program contain one default thread known as main thread and this thread is present in main method and default priority of this thread is 5 or normal priority

```
public class MTAPP
{
    public static void main(String x[])
    {
        Thread t=Thread.currentThread();
        String tname=t.getName();
        System.out.println("This is the default thread name "+tname);
        int priority=t.getPriority();
        System.out.println("Thread priority is  "+priority);
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac MTAPP.java

C:\Program Files\Java\jdk-24\bin>java MTAPP
This is the default thread name main
Thread priority is  5
```

Q. What is the Thread Priority?

Thread priority is used for decide when thread should start its execution called as thread priority

There are three types of priority in Thread

Constant name	Value of constant	Description of priority
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Default priority or normal priority
Thread.MAX_PRIORITY	10	Highest priority

Note: for manage priority internally Thread class has integer static variable with default values shown in following code snippet

```
class Thread
{
    public static int MAX_PRIORITY=10;
    public static int MIN_PRIORITY=1;
    public static int NORM_PRIORITY=5;
}
```

If we want to apply thread priority on thread or retrieve priority of thread we have following methods

1. **void setPriority(int priority):** this method is used for set priority on thread and priority value should be between 1 to 10
2. **int getPriority():** this method can return priority of thread or current priority of thread

Example with source code

```
import java.util.*;
class Table
{
 synchronized void  showTable(int no)
  {
      try{
              for(int i=1; i<=10; i++)
                    {
                      System.out.printf("%d X %d = %d\n",i,no,i*no);
                            if(i==5)
                            { wait();
                            }
                            Thread.sleep(1000);
                    }
             }
             catch(Exception ex)
             { System.out.println("Error is "+ex);
             }
 }
```

```
  public synchronized void restart(){

            try{
                    notifyAll();
            }
            catch(Exception ex)
            {System.out.println("Error is "+ex);
            }
  }
}
class Two extends Thread
{  Table table;
   Two(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(2);
   }
}
class Three extends Thread
{   Table table;
   Three(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(3);
   }
}
class Four extends Thread
{   Table table;
   Four(Table table)
   {  this.table=table;
   }
   public void run()
   {   table.showTable(4);
   }
}
public class SyncAsyncApp
{
    public static void main(String x[])throws Exception
          {
              Table t = new Table();
                    Two tw = new Two(t);
                    Three th= new Three(t);

                    Four f = new Four(t);
```

```
                f.setPriority(Thread.MIN_PRIORITY);
                tw.setPriority(Thread.MAX_PRIORITY);
                th.setPriority(Thread.NORM_PRIORITY);

                tw.start();
                th.start();
                f.start();

                do{
                        Scanner xyz =new Scanner(System.in);
                        String msg=xyz.nextLine();
                        if(msg.equals("restart"))
                        { t.restart();
                        }
                        if(msg.equals("stop"))
                        {System.exit(0);
                        }
                }while(true);
        }
}
```

Q. What is a Daemon thread?

Daemon thread is known as a service provider thread which means this type of thread executes in the background and provides service to other threads.
E.g Garbage collector

Note: User can create own thread as Daemon thread and for that purpose user has methods

1. **void setDaemon(boolean): this method is used for mark user thread as daemon thread if we pass true value in it**
2. **boolean isDaemon():** this method is used for check user thread is daemon thread or not if user thread is daemon thread return true otherwise return false

Example with source code

```
class ABC extends Thread
{  static int count=0;
   public void run()
   {
     try{
            for(int i=1; i<=5; i++)
                { System.out.printf("I is %d\n",i);
                        Thread.sleep(1000);
                        ++count;
                }
          }
          catch(Exception ex)
```

```
        { System.out.println("Error is "+ex);
        }
  }
}
public class ABCAPP
{
   public static void main(String x[])throws Exception
        {       ABC a1 = new ABC();
                a1.setDaemon(true);
                a1.start();
        }
}
```

How to create thread using a Runnable interface

Q. Why do we need to create a thread using Runnable if we can create a thread using Thread class?

Some time if we want to implement the thread using Thread class so we need to inherit Thread class in user define class but if there is already other class inherited then we cannot inherit the Thread class in user define class because of diamond problem and if we want to solve this problem we can use Runnable interface for thread implementation purpose

```
class B
{
}
class A   extends B ,Thread
{



}
```

Note: here we want to create class A as Thread so class A has parent class name as B and we try to inherit Thread class in class A but in java we cannot give two classes as parent to single class so we can use Runnable interface for solve this problem shown in following code

```
class B
{
}
class A   extends B  implements Runnable
{



}
```

Steps to implement thread using a Runnable

---

a. **Add java.lang package**
   import java.lang.*
b. **Create user define class and implements Runnable interface in it**

   ---

   class Test implements Runnable {

```
}
```

c. Override its run() method and write thread logics

```
class Test implements Runnable
{
        public void run()
        {
                try{

                        for(int i=1; i<=5; i++)
                        {    System.out.printf("I = %d\n",i);
                             Thread.sleep(3000);
                        }
                }catch(Exception ex)
                 { System.out.println("Error is "+ex);
                 }
        }
}
```

d. Create object of Runnable implementer class

```
Test t = new Test();
```

e. Create object of Thread class and use its given constructor

Thread(Runnable): According to dynamic polymorphism we can pass Runnable reference in it using upcasting or pass implementer class object of Runnable directly

Example:

```
Runnable r = new Test();
Thread thread=new Thread(r);
```

f. Call start() method of Thread for start thread

```
class Test implements Runnable
{
        public void run()
        {
                try{

                        for(int i=1; i<=5; i++)
                        {
                                System.out.printf("I = %d\n",i);
                                Thread.sleep(3000);
                        }
                }catch(Exception ex)
                 { System.out.println("Error is "+ex);
```

```
                }
            }
        }
        public class TestRunnableApp
        {
            public static void main(String x[])
            {
                Runnable r = new Test();
                    Thread thread=new Thread(r);
                    thread.start();
            }
        }
```

g. Use Thread class other methods for manage thread

Note: you can create thread by using anonymous inner class

```
public class TestRunnableApp
{
    public static void main(String x[])
    {
        Runnable r = new Runnable(){

            public void run(){
                try{
                    for(int i=1; i<=5; i++)
                    { System.out.printf("I =%d\n",i);
                        Thread.sleep(1000);
                    }
                }
                catch(Exception ex)
                { System.out.println("error is  "+ex);
                }
            }
        };
        Thread t = new Thread(r);
        t.start();
    }
}
```

Or

```
public class TestRunnableApp
{
    public static void main(String x[])
    {
        Runnable r =  ()->{
                try{
                    for(int i=1; i<=5; i++)
```

```
                                        { System.out.printf("I =%d\n",i);
                                                Thread.sleep(1000);
                                        }
                                }
                                catch(Exception ex)
                                { System.out.println("error is  "+ex);
                                }
                        };
                Thread t = new Thread(r);
                t.start();
        }
}
```

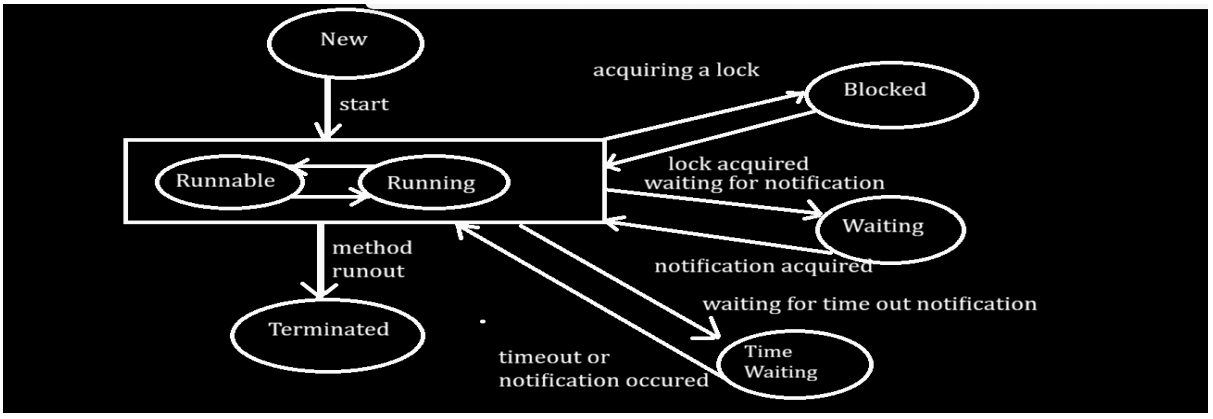
Or

```
public class TestRunnableApp
{
        public static void main(String x[])
        {
                new Thread(()->{
                                try{
                                        for(int i=1; i<=5; i++)
                                        { System.out.printf("I =%d\n",i);
                                                Thread.sleep(1000);
                                        }
                                }
                                catch(Exception ex)
                                { System.out.println("error is  "+ex);
                                }
                        }).start();
        }
}
```

Thread Life cycle

Thread life cycle define or decide how thread will created in memory and define states of thread in application as well as decide how thread will destroy called as thread life cycle

Following diagram demonstrate life cycle of thread



1. **New** : thread is in new state when its object is created but has not yet started its execution so its code has not begun running
   Syntax: public static final Thread.State.NEW
2. **Runnable:** Thread state for a runnable thread. A Thread in the runnable state is executing in the Java Virtual Machine but it may be waiting for other resources from operating system such as processor
   Syntax: public static final Thread.state.Runnable
3. **Blocked State:** A thread is in the blocked state when it is waiting to acquire lock that is currently held by another thread
4. **Waiting:** Thread is in the waiting state due to calling one of the following methods

   Object.wait - with no timeout - wait()
   Thread.join(): with time timeout - join()

5. **Timed waiting: A thread is in timed waiting state due to calling one of the following method**

   void wait(int milliseconds) : method from object class
   void sleep(int milliseconds): method from Thread class
   void join(int milliseconds): method from thread class

   Syntax: public static final Thread.State.TIMED_WAITING
6. Terminated: termination means thread complete its execution using a following cases
   a. Successfully complete all task
   b. Developer terminate manually
   c. Exception occur at runtime

   Note: developer can call stop() method and can terminate thread manually

Syntax: public static final Thread.State.TERMINATED

Example with source code

```
import java.util.*;
class Table
{  static Thread t;
  synchronized void  showTable(int no)
  {
      try{
            for(int i=1; i<=10; i++)
                  {
                    System.out.printf("%d X %d = %d\n",i,no,i*no);
                          if(i==5)
                          {
                      wait(10000);
                            System.out.println("Now two thread  "+t.getState());//two.getSTate()
                          }
                          Thread.sleep(1000);
                  }
```

```
            }
            catch(Exception ex)
            { System.out.println("Error is "+ex);
            }
 }
 public synchronized void restart(){

           try{
                    notifyAll();
           }
           catch(Exception ex)
           {System.out.println("Error is "+ex);
           }
 }
}
class Two extends Thread
{  Table table;
   Two(Table table)
   {  this.table=table;
   }
   public void run()
   {   System.out.println("Now two thread  "+this.getState());
       Table.t=this;
     table.showTable(2);
   }
}

public class SyncAsyncApp
{   public static void main(String x[])throws Exception
        {   Table t = new Table();
                Two tw = new Two(t);
                System.out.println("Now two thread in  "+tw.getState());//new
                tw.start();
             Thread.sleep(6000);
                System.out.println("Now two thread in "+Table.t.getState());
                Thread.sleep(20000);
                        System.out.println("Now two thread in "+Table.t.getState());

        }
}
```